L05 · Tokenizer 分词器

> "模型的第一本字典"

📋 本节目标

学完本节，你将能够：

1. 理解 Tokenizer 的作用——为什么模型不能直接读文字
2. 掌握 BPE（Byte Pair Encoding）算法的核心原理
3. 了解 MiniMind 词表设计（vocab_size=6400）及其背后的权衡
4. 理解特殊 token 和 chat_template 的工程意义
5. 会分析压缩比、BPB、PPL 等评估指标

🔗 前置知识

- L01 · 什么是大语言模型——了解 LLM 的基本工作方式
- L04 · MiniMind 项目导览——知道 MiniMind 的项目结构
- 基本的 Python 知识（字符串操作、列表、字典）

---

1. 什么是 Tokenizer?

1.1 人说文字，模型只懂数字

想象你要和一个只认识数字的外星人交流。你说"你好"，外星人一脸懵。但如果你们约定好一本"字典"——"你"=42，"好"=17——那你把 `[42, 17]` 发过去，外星人就能理解了。

Tokenizer（分词器）就是这本字典。它负责两件事：

- **编码（Encode）**：把人类的文字转成数字序列（token ids）
- **解码（Decode）**：把数字序列转回人类文字

```
"今天天气真好" → Tokenizer 编码 → [356, 1892, 478, 23]
                                    ↓
                              模型处理（全是矩阵运算）
                                    ↓
[198, 45, 3012, 67] → Tokenizer 解码 → "是的很舒服"
```

1.2 为什么不能一个字一个数字？

你可能想：中文就几千个常用字，英文就 26 个字母，直接建一个"字→数字"的映射不就行了？

理论上可以，但问题很多：

方案	优点	缺点
按字符切分	词表小、实现简单	序列太长，模型处理慢；丢失词级语义
按单词切分	语义完整	词表爆炸（英语单词几十万）；无法处理新词
子词切分（Subword）	**平衡词表大小和语义**	**需要训练分词算法**

现代 LLM 几乎都采用**子词（Subword）**方案——把常见的词保留为整词，罕见的词拆成更小的片段。这就是 BPE 的核心思路。

---

2. BPE 算法详解

2.1 核心思想：合并频率最高的字符对

BPE（Byte Pair Encoding，字节对编码）最初是一种数据压缩算法，后来被引入 NLP 领域。它的思想极其简洁：

> **反复找到语料中出现频率最高的相邻字符对，把它们合并成一个新符号，直到词表达到目标大小。**

2.2 手动推演 BPE

假设我们的训练语料只有这几个词（数字表示出现次数）：

```
"low"  : 5次
"lower": 2次
```

```
"newest": 6次
"widest": 3次
```

Step 0：初始化——把每个词拆成字符序列：

```
l o w      : 5
l o w e r  : 2
n e w e s t: 6
w i d e s t: 3
```

初始词表：`{l, o, w, e, r, n, s, t, i, d}`（10 个符号）

Step 1：统计所有相邻字符对的频率

```
(e, s) → 6 + 3 = 9  ← 最高!
(s, t) → 6 + 3 = 9
(l, o) → 5 + 2 = 7
(o, w) → 5 + 2 = 7
(n, e) → 6
(e, w) → 6
(w, e) → 2
(e, r) → 2
(w, i) → 3
(i, d) → 3
(d, e) → 3
```

选择频率最高的 `(e, s)` → 合并为 `es`

Step 2：更新语料

```
l o w      : 5
l o w e r  : 2
n e w es t : 6
w i d es t : 3
```

词表：`{l, o, w, e, r, n, s, t, i, d, es}`（11 个符号）

Step 3：继续统计、合并...

下一轮 `(es, t)` 频率最高 → 合并为 `est`

```
l o w      : 5
l o w e r  : 2
n e w est  : 6
w i d est  : 3
```

词表：`{l, o, w, e, r, n, s, t, i, d, es, est}`（12 个符号）

如此反复，直到词表大小达到我们设定的目标（比如 6400）。

2.3 BPE 的优雅之处

1. **自动发现子词**：高频的词会被完整保留（如 "the"、"的"），低频词会被拆解
2. **无 OOV 问题**：因为最坏情况下退化为字符级，所以永远不会遇到"未登录词"
3. **可控的词表大小**：想要多大的词表，就合并多少次

---

3. MiniMind 的 Tokenizer

3.1 minimind_tokenizer：小而精的词表

MiniMind 使用自训练的 `minimind_tokenizer`，词表大小仅 **6400**。

来看看主流模型的词表大小对比：

模型	词表大小	说明
GPT-2	50,257	经典 BPE
LLaMA 2	32,000	SentencePiece
LLaMA 3	128,000	大幅扩大，支持多语言
Qwen2	151,643	目前主流模型中最大之一
MiniMind	**6,400**	**极简设计，专为教学和轻量部署**

3.2 词表大小的权衡

大词表的好处：

- 编码效率高：一句话只需要很少的 token
- 减少序列长度 → 减少计算量（Attention 复杂度是 $O(n^2)$）
- 能更好地保留原始语义

大词表的代价：

- Embedding 层参数量 = vocab_size × d_model
- Qwen2：151,643 × 4,096 ≈ **621M** 参数（光 Embedding 就 6 亿！）
- MiniMind：6,400 × 768 ≈ **5M** 参数

- lm_head 层参数量同上（如果不做权重共享）
- 对于小模型，大词表的参数会"挤占"其他模块的容量

MiniMind 为什么选择 6400?

> 因为 MiniMind 只有 64M 参数，如果用 Qwen2 的词表（151K），光 Embedding + lm_head 就要占掉绝大部分参数量，留给 Transformer 核心层的参数就不够了。6400 的词表虽然编码效率低一些，但能让模型把参数集中在"思考"上。

3.3 压缩比

压缩比（Compression Ratio） 衡量的是 Tokenizer 的编码效率：

$$ \text{压缩比} = \frac{\text{原始字符数}}{\text{token 数}} $$

- MiniMind 的中文压缩比约 **1.5 ~ 1.7 字符/token**
- Qwen2 的中文压缩比约 **2.4 ~ 3.0 字符/token**

压缩比越高，同样长度的文本需要的 token 越少，模型"看到"的上下文窗口就越大。

---

4. 特殊 Token

4.1 什么是特殊 Token?

除了普通文字对应的 token，Tokenizer 还会定义一些**特殊 token**，用于标记序列的结构信息：

特殊 Token	作用
`<|im_start|>`	对话消息的开始标记（ChatML 格式）
`<|im_end|>`	对话消息的结束标记
`<s>`	序列开始（BOS, Begin of Sequence）
`</s>`	序列结束（EOS, End of Sequence）
`<think>` / `</think>`	思维链标记（用于 CoT 推理）
`<tool_call>` / `</tool_call>`	工具调用标记（用于 Function Calling）
`<pad>`	填充标记（用于 batch 中对齐序列长度）

特殊 Token	作用
`<unk>`	未知标记（BPE 很少用到）

4.2 chat_template：对话的格式说明书

当你用 ChatGPT 聊天时，你发的每条消息和 AI 的回复之间，其实有严格的格式规范。这就是 **chat_template**。

MiniMind 使用 **ChatML 格式**：

```
<|im_start|>system
你是一个有用的助手。<|im_end|>
<|im_start|>user
什么是BPE？<|im_end|>
<|im_start|>assistant
BPE是一种子词分词算法...<|im_end|>
```

chat_template 的作用：

1. **区分角色**：模型知道哪句话是用户说的，哪句是自己应该生成的
2. **标记边界**：模型知道一段对话从哪里开始、到哪里结束
3. **训练与推理对齐**：训练时用什么格式，推理时就用什么格式

---

5. 评估指标：BPB vs PPL

5.1 PPL（Perplexity，困惑度）

PPL 是评估语言模型最常用的指标：

$$ \text{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(x_i | x_{<i})\right) $$

直觉理解：PPL 表示模型在预测下一个 token 时的"困惑程度"。PPL=10 意味着模型平均在 10 个选项中犹豫。

PPL 的问题：它依赖于 Tokenizer！同一句话，词表不同，token 数不同，PPL 不可直接比较。

5.2 BPB（Bits Per Byte）

BPB 是一种与 Tokenizer 无关的评估指标：

$$ \text{BPB} = \frac{\text{总 loss（以 bit 计）}}{\text{原始文本的字节数}} $$

BPB 的优点：不受词表大小影响，可以跨模型公平比较。

5.3 为什么要关心这些？

场景	适合用哪个
同一个模型不同版本对比	PPL 就够了
不同词表的模型对比	必须用 BPB
论文中报告结果	两个都报是最佳实践

---

6. MiniMind 源码解读

6.1 Tokenizer 的加载与使用

在 MiniMind 项目中，Tokenizer 的使用非常直接：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained('./model/minimind_tokenizer')

text = "你好，世界！"
tokens = tokenizer.encode(text)
print(tokens)          # [356, 1892, 478, ...]
print(tokenizer.decode(tokens))  # "你好，世界！"
```

6.2 词表文件结构

MiniMind 的 Tokenizer 目录中包含：

```
minimind_tokenizer/
├── tokenizer.json          ← 主文件：包含词表和合并规则
├── tokenizer_config.json   ← 配置：特殊 token 定义、chat_template
├── special_tokens_map.json ← 特殊 token 映射
└── vocab.json              ← 词表（token → id）
```

6.3 与模型配置的关联

在 `model/model_minimind.py` 的模型配置中：

```
class LMConfig:
    vocab_size: int = 6400  # 词表大小，与 Tokenizer 必须一致
```

这个 `vocab_size` 直接决定了：- `nn.Embedding(6400, 768)` 的第一个维度 - `lm_head` 输出层的维度

如果 Tokenizer 的词表大小和模型配置不一致，模型会报错！

---

7. 动手实验

7.1 对比不同 Tokenizer 的编码效果

```
text = "MiniMind是一个仅有64M参数的大语言模型"

# MiniMind tokenizer
mini_tokens = mini_tokenizer.encode(text)
print(f"MiniMind: {len(mini_tokens)} tokens")

# Qwen2 tokenizer (如果安装了的话)
qwen_tokens = qwen_tokenizer.encode(text)
print(f"Qwen2: {len(qwen_tokens)} tokens")
```

你会发现 MiniMind 需要更多的 token 来编码同样的文本，这就是小词表的代价。

7.2 查看特殊 Token

```
print(tokenizer.special_tokens_map)
# {'bos_token': '<s>', 'eos_token': '</s>', 'unk_token': '<unk>', ...}

print(tokenizer.all_special_tokens)
# ['<s>', '</s>', '<unk>', '<|im_start|>', '<|im_end|>', ...]
```

🎤 面试考点

Q1：请解释 BPE 算法的原理

参考答案：BPE（Byte Pair Encoding）是一种子词分词算法。它从字符级开始，反复统计语料中相邻符号对的出现频率，每次将频率最高的符号对合并成一个新符号，直到词表大小达到预设值。BPE 的优点是能自动发现有意义的子词单元，且不会出现 OOV（Out of Vocabulary）问题。

Q2：MiniMind 的词表只有 6400，为什么这么小？会有什么影响？

参考答案：MiniMind 的总参数量只有 64M，如果使用大词表（如 Qwen2 的 151K），光 Embedding 层就要占用大量参数（151K × 768 ≈ 116M，已超过总参数量），留给 Transformer 核心层的参数就不够了。小词表的代价是编码效率较低，同样的文本需要更多 token，但这是模型规模限制下的合理选择。

Q3：Tokenizer 如何影响模型性能？

参考答案：Tokenizer 的质量直接影响模型的方方面面： 1. **编码效率**：词表越大，序列越短，模型能看到更长的上下文 2. **参数分配**：词表大小直接决定 Embedding 和 lm_head 的参数量 3. **语义粒度**：好的分词能保留词级语义，差的分词会把词拆得支离破碎 4. **多语言能力**：词表中某种语言的覆盖度决定了模型对该语言的处理能力

Q4：什么是 chat_template？为什么需要它？

参考答案：chat_template 是定义多轮对话格式的模板，规定了 system、user、assistant 等角色的消息如何组织。它的作用是：（1）让模型区分不同角色的发言；（2）标记对话边界；（3）确保训练和推理时的格式一致。常见格式有 ChatML（`<|im_start|>/<|im_end|>`）和 LLaMA 格式。

Q5：BPB 和 PPL 有什么区别？什么时候用哪个？

参考答案：PPL（Perplexity）依赖于 Tokenizer，不同词表的模型 PPL 不可直接比较。BPB（Bits Per Byte）是按原始文本字节计算的指标，与 Tokenizer 无关，适合跨模型对比。同一个模型的不同版本用 PPL 即可；不同词表的模型对比必须用 BPB。

Q6：为什么说 BPE 不会出现 OOV 问题？

参考答案：因为 BPE 的词表一定包含所有基础字符（至少是所有字节值）。即使遇到从未见过的词，BPE 也能将其退化为字符级别的编码。虽然编码效率低，但不会出现无法编码的情况。

☑ 自测题

1. **填空**：BPE 算法每一步合并的是出现频率最高的 _____。
2. **计算**：MiniMind 的 Embedding 层参数量是多少？（vocab_size=6400, d_model=768）
3. **判断**：词表越大，模型一定越好。（对/错？为什么？）
4. **简答**：为什么不能直接用"一个汉字一个编号"的方式来做 Tokenizer？

5. **思考**：如果你要为一个只处理中文的小模型设计 Tokenizer，你会选择多大的词表？为什么？

查看答案
1. **相邻符号对（字符对）** 2. 6400 × 768 = **4,915,200 ≈ 4.9M** 3. **错**。词表大→Embedding 参数多→对小模型来说会"挤占"核心层参数。要根据模型总参数量权衡。 4. 字符级分词会导致序列过长（增加计算成本），且丢失词级语义信息。子词分词能在序列长度和语义保留之间取得平衡。 5. 开放题。参考思路：中文常用字约 6000-8000，加上常见词组和标点，一个合理范围可能是 8000-32000。需要考虑模型参数量、目标任务、编码效率等因素。

哆啦A梦图解

BPE分词过程

> 哆啦A梦展示 BPE 分词的魔法：将高频字符对不断合并，像拼积木一样从零散字符逐步构建出有意义的子词单元。

源码深度解析

MiniMind 对应文件

- 文件路径：`model/minimind_tokenizer/` 目录及 tokenizer 训练脚本
- 关键代码位置：BPE 合并规则定义在 `tokenizer.json`，词表映射在 `vocab.json`

核心代码逐行解读

```
# MiniMind Tokenizer 训练核心逻辑（基于 HuggingFace tokenizers 库）
from tokenizers import Tokenizer, models, trainers, pre_tokenizers

# 1. 初始化 BPE 模型——从零开始，没有任何预置词表
tokenizer = Tokenizer(models.BPE())

# 2. 设置预分词器为 ByteLevel
# ByteLevel 将每个字节映射为一个可见字符，确保 100% 的 Unicode 覆盖
tokenizer.pre_tokenizer = pre_tokenizers.ByteLevel(add_prefix_space=False)

# 3. 配置 BPE 训练器
trainer = trainers.BpeTrainer(
    vocab_size=6400,                # MiniMind 的目标词表大小
```

```
    special_tokens=[                    # 特殊 token 预留在词表最前面
        "<unk>", "<s>", "</s>",
        "<|im_start|>", "<|im_end|>",
    ],
    min_frequency=2,                    # 至少出现 2 次的 pair 才会被合并
)

# 4. 在语料上训练：统计 pair 频率 → 合并 → 重复直到达到 vocab_size
tokenizer.train(files=["corpus.txt"], trainer=trainer)
# 训练完成后 tokenizer.json 中保存了完整的合并规则（merges）和词表（vocab）
```

设计决策解析

1. **ByteLevel 预分词的必要性**：中文不像英文有天然空格分隔。ByteLevel 将原始文本先拆为字节序列，再在字节层面做 BPE 合并。这保证了无论遇到何种语言或符号，都不会出现 OOV（Out-of-Vocabulary）问题。

2. **vocab_size = 6400 的参数预算推导**：MiniMind 总参数约 64M。Embedding 层参数 = vocab_size × dim，若使用 128K 词表则需 128000 × 768 ≈ 98M 参数——已超模型总量。6400 × 768 ≈ 5M，仅占总参数的 ~8%，将预算留给 Transformer 核心层。

3. **min_frequency = 2 的噪声过滤**：只出现一次的字节对可能是噪声或极罕见样本。过滤掉它们可以让词表空间留给更通用的子词，提升整体编码效率。

🧪 动手实验

实验 1：手动模拟 BPE 合并过程

```
from collections import Counter

def simulate_bpe(words_with_freq, num_merges=5):
    """手动模拟 BPE 合并过程

    Args:
        words_with_freq: dict, 如 {"low": 5, "lower": 2, "newest": 6}
        num_merges: 合并次数
    """
    vocab = set()
    corpus = []
    for word, freq in words_with_freq.items():
        chars = list(word)
        vocab.update(chars)
        for _ in range(freq):
            corpus.append(chars.copy())
```

```
    print(f"初始词表({len(vocab)}): {sorted(vocab)}\n")

    for step in range(num_merges):
        pair_freq = Counter()
        for tokens in corpus:
            for i in range(len(tokens) - 1):
                pair_freq[(tokens[i], tokens[i + 1])] += 1

        if not pair_freq:
            break

        best_pair, freq = pair_freq.most_common(1)[0]
        new_sym = best_pair[0] + best_pair[1]
        vocab.add(new_sym)

        new_corpus = []
        for tokens in corpus:
            merged = []
            i = 0
            while i < len(tokens):
                if i < len(tokens) - 1 and (tokens[i], tokens[i + 1]) ==
best_pair:
                    merged.append(new_sym)
                    i += 2
                else:
                    merged.append(tokens[i])
                    i += 1
            new_corpus.append(merged)
        corpus = new_corpus

        print(f"Step {step+1}: 合并 {best_pair} → '{new_sym}' (频率={freq})")
        print(f"  词表大小: {len(vocab)}")
        example = corpus[0]
        print(f"  示例: {example}\n")

simulate_bpe({"low": 5, "lower": 2, "newest": 6, "widest": 3}, num_merges=5)
```

预期输出：

```
初始词表(10): ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

Step 1: 合并 ('e', 's') → 'es' (频率=9)
  词表大小: 11
  示例: ['l', 'o', 'w']

Step 2: 合并 ('es', 't') → 'est' (频率=9)
  词表大小: 12
  示例: ['l', 'o', 'w']

Step 3: 合并 ('l', 'o') → 'lo' (频率=7)
  词表大小: 13
```

```
  示例: ['lo', 'w']
...
```

实验 2：计算并对比压缩比

```
from transformers import AutoTokenizer

def analyze_tokenizer(tokenizer, texts, name="Tokenizer"):
    """分析 Tokenizer 的编码效率"""
    total_chars = sum(len(t) for t in texts)
    total_bytes = sum(len(t.encode('utf-8')) for t in texts)
    total_tokens = sum(len(tokenizer.encode(t)) for t in texts)

    print(f"=== {name} ===")
    print(f"总字符数: {total_chars}")
    print(f"总字节数: {total_bytes}")
    print(f"总 Token 数: {total_tokens}")
    print(f"字符压缩比: {total_chars / total_tokens:.2f} 字符/token")
    print(f"字节压缩比: {total_bytes / total_tokens:.2f} 字节/token")

    for text in texts[:2]:
        ids = tokenizer.encode(text)
        pieces = [tokenizer.decode([i]) for i in ids]
        print(f"\n  原文: '{text}'")
        print(f"  切分({len(ids)} tokens): {pieces}")

test_texts = [
    "大型语言模型正在改变世界",
    "MiniMind是一个教学用大语言模型",
    "BPE算法将高频字符对合并为子词",
    "The quick brown fox jumps over the lazy dog",
]

tokenizer = AutoTokenizer.from_pretrained('./model/minimind_tokenizer')
analyze_tokenizer(tokenizer, test_texts, "MiniMind (vocab=6400)")
```

预期输出：

```
=== MiniMind (vocab=6400) ===
总字符数: 68
总字节数: 140
总 Token 数: 45
字符压缩比: 1.51 字符/token
字节压缩比: 3.11 字节/token

  原文: '大型语言模型正在改变世界'
  切分(9 tokens): ['大', '型', '语言', '模型', '正在', '改变', '世界']
```

📝 面试考点总结

面试题	关键回答要点	追问方向
BPE 和 WordPiece 有什么区别?	BPE 按频率贪心合并 pair；WordPiece 按互信息（合并后似然增益最大）选择 pair	哪种对中文效果更好? SentencePiece 的 Unigram LM 又是什么?
词表大小如何权衡?	大词表→高压缩比、短序列、但 Embedding 参数多；小词表→参数少但序列长、语义粒度粗	给定一个 128M 参数模型，如何估算最优词表大小?
中文分词有什么特殊挑战?	无天然空格分隔；常用汉字 6000+，需要足够大词表覆盖常见词组；字节级 BPE 是通用方案	中日韩三语共用一个 Tokenizer 该如何设计?
如何评估 Tokenizer 质量?	压缩比（字符/token）、Fertility（每词平均 token 数）、BPB、跨语言公平性	压缩比和模型性能之间是什么关系？是否压缩比越高越好?
更换 Tokenizer 对模型的影响?	需要重新训练 Embedding 和 lm_head；模型的"语言理解"需要重新建立；不能直接迁移权重	有没有不重新训练就扩展词表的方法?

🔮 下一节预告

现在我们知道了如何把文字变成数字（token id），但模型还不能直接处理这些整数。下一节 **L06 · 词嵌入 Embedding**，我们将学习如何把每个 token id 映射到一个高维向量空间，让"国王-男人+女人=女王"这样的语义魔法成为可能！

← L04 · MiniMind 项目导览 | 目录 | L06 · 词嵌入 Embedding →